

Autori:

- Boccaleoni Marco
- Magagnin Gilberto
- Pizzolli Simone

Docente:

- Prof. Rigon Riccardo

Analisi Idrologica e Curve di Probabilità Pluviometrica

Definizione dei carichi idrologici di progetto per il bacino di Romeno

1986–2021

Periodo di registrazione

32

Anni di osservazioni
valide

5

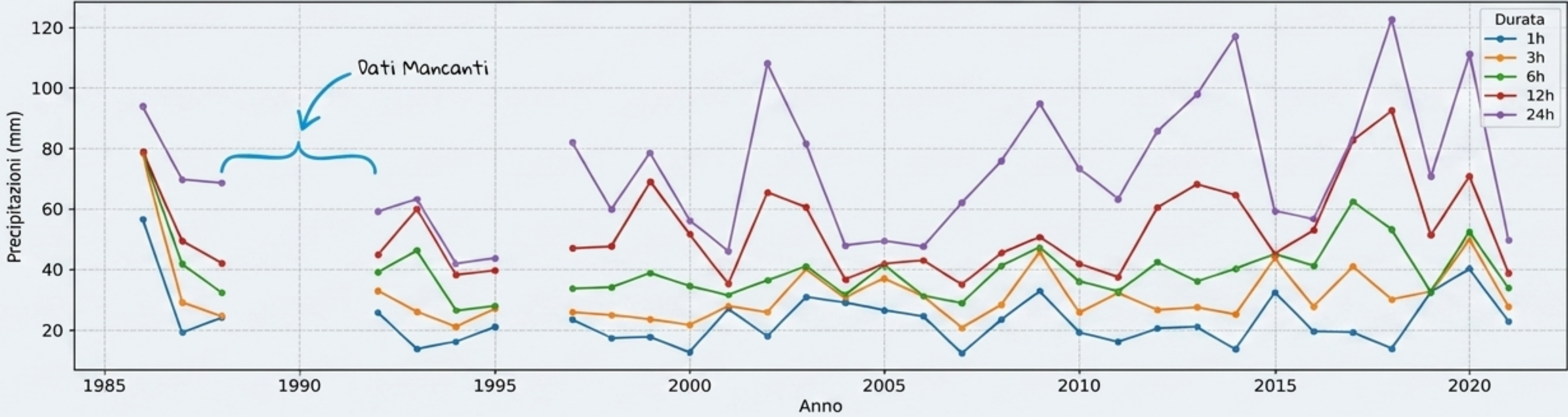
Durate analizzate:
1h, 3h, 6h, 12h, 24h

Gestione Dati: NaN Mantenuti

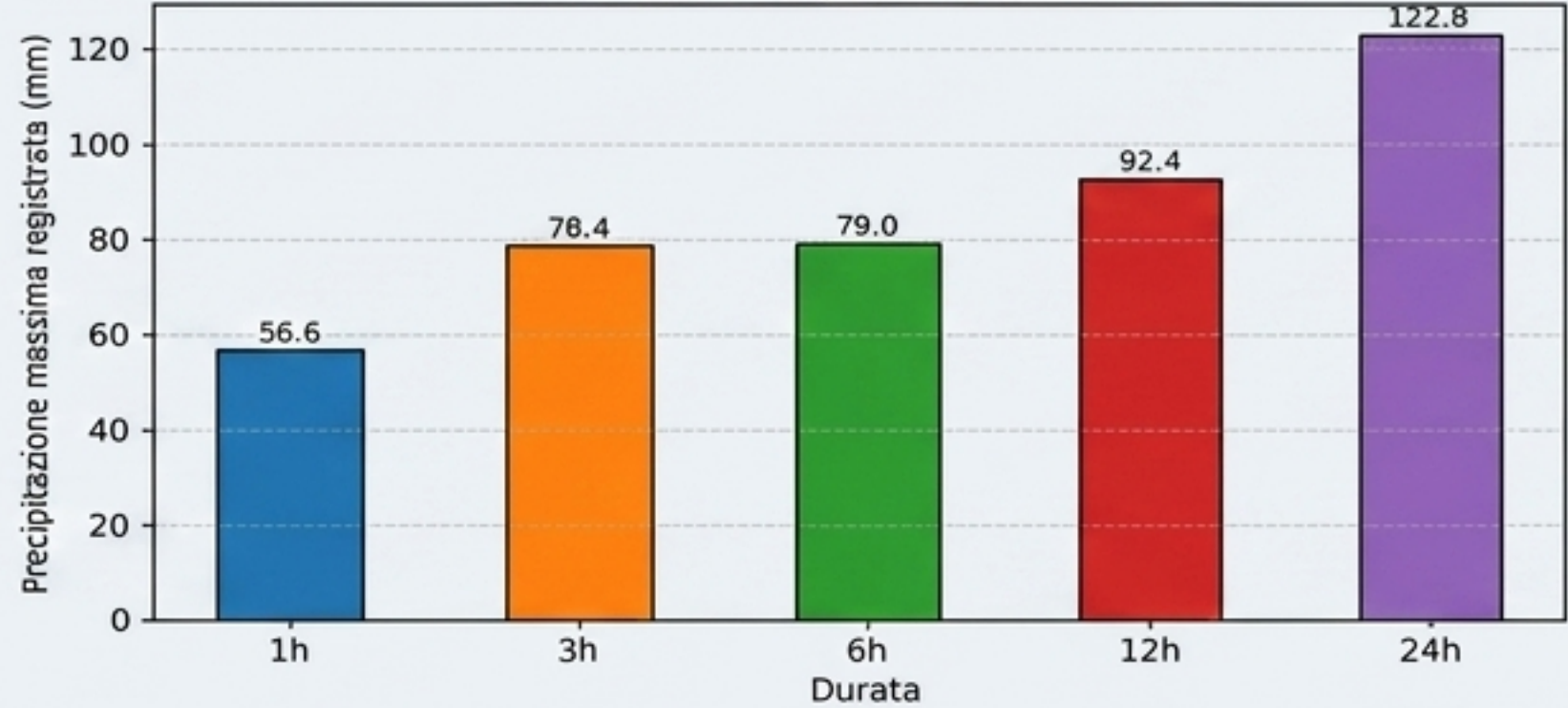
4 anni mancanti (1989–1991, 1996) e 72 righe vuote precedenti al 1986 sono stati isolati. Nessuna interpolazione artificiale per preservare la fedeltà statistica.

Evoluzione Storica e Massimi Assoluti

Andamento storico delle piogge massime - Romeno



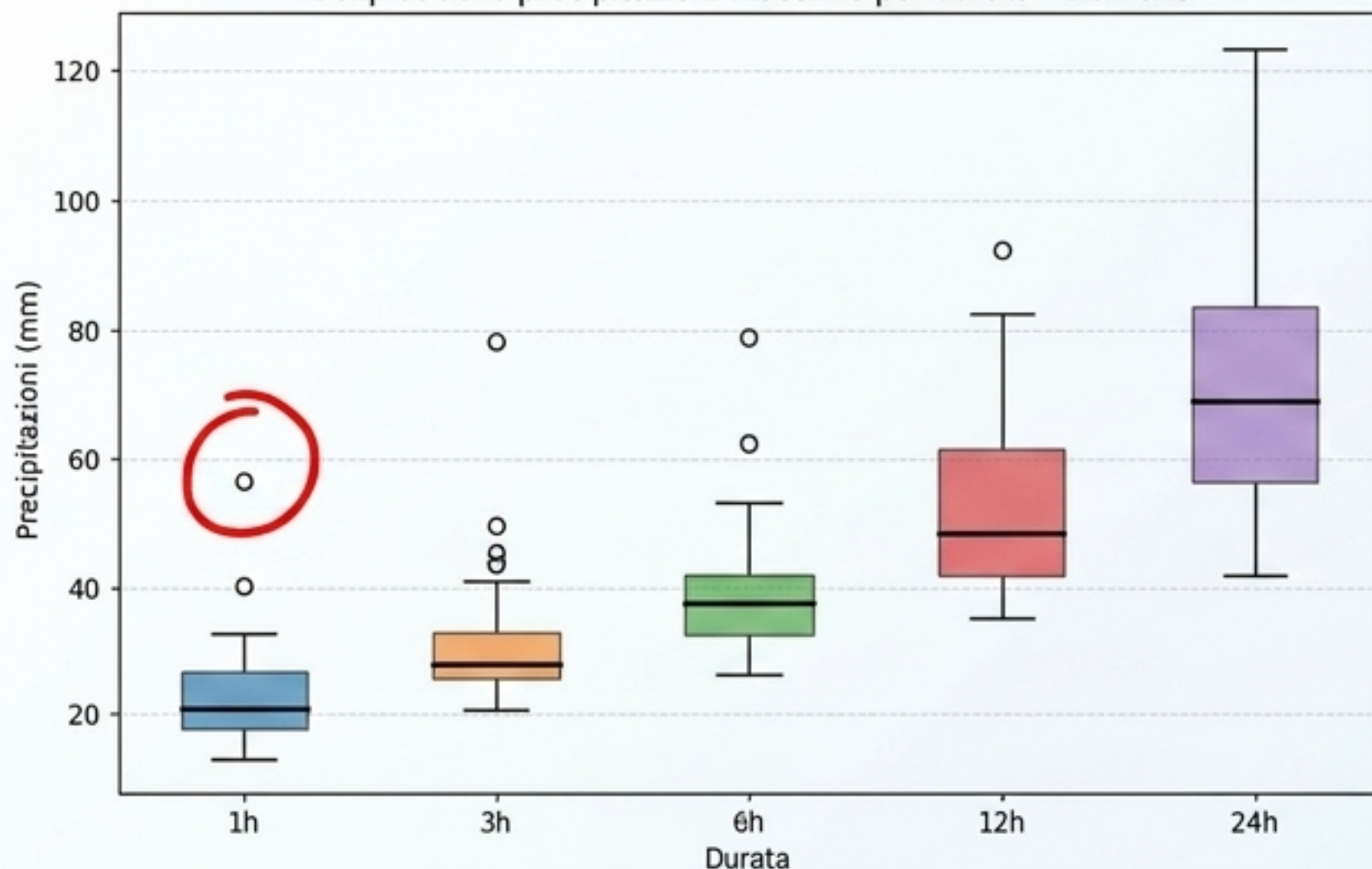
Massimo storico per durata - Romeno



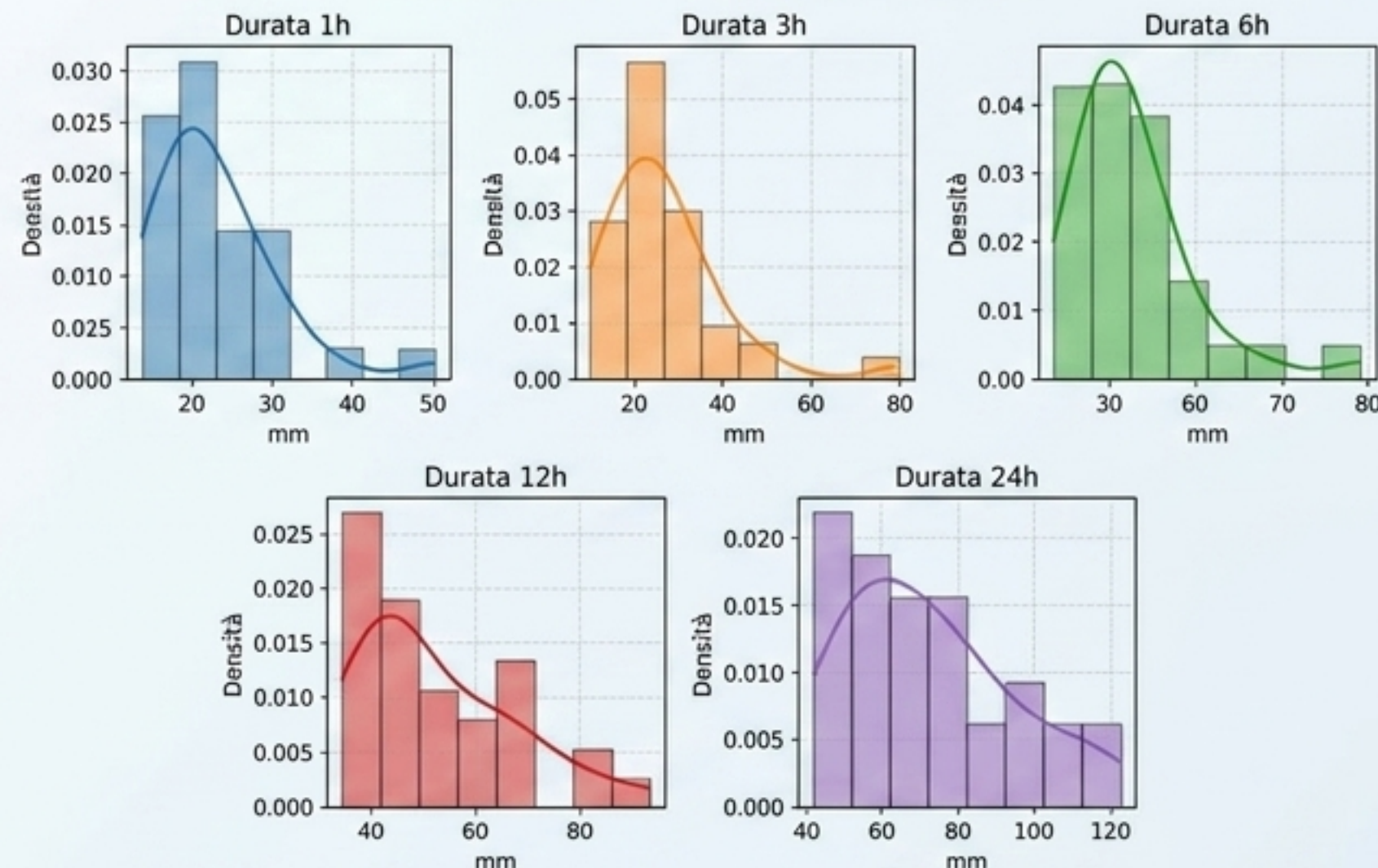
Progressione fisica rispettata: l'altezza di pioggia cresce sistematicamente con la durata.
Picco assoluto: 122.8 mm in 24h (2018).

L'Outlier del 1986 e l'Asimmetria

Boxplot delle precipitazioni massime per durata - Romeno

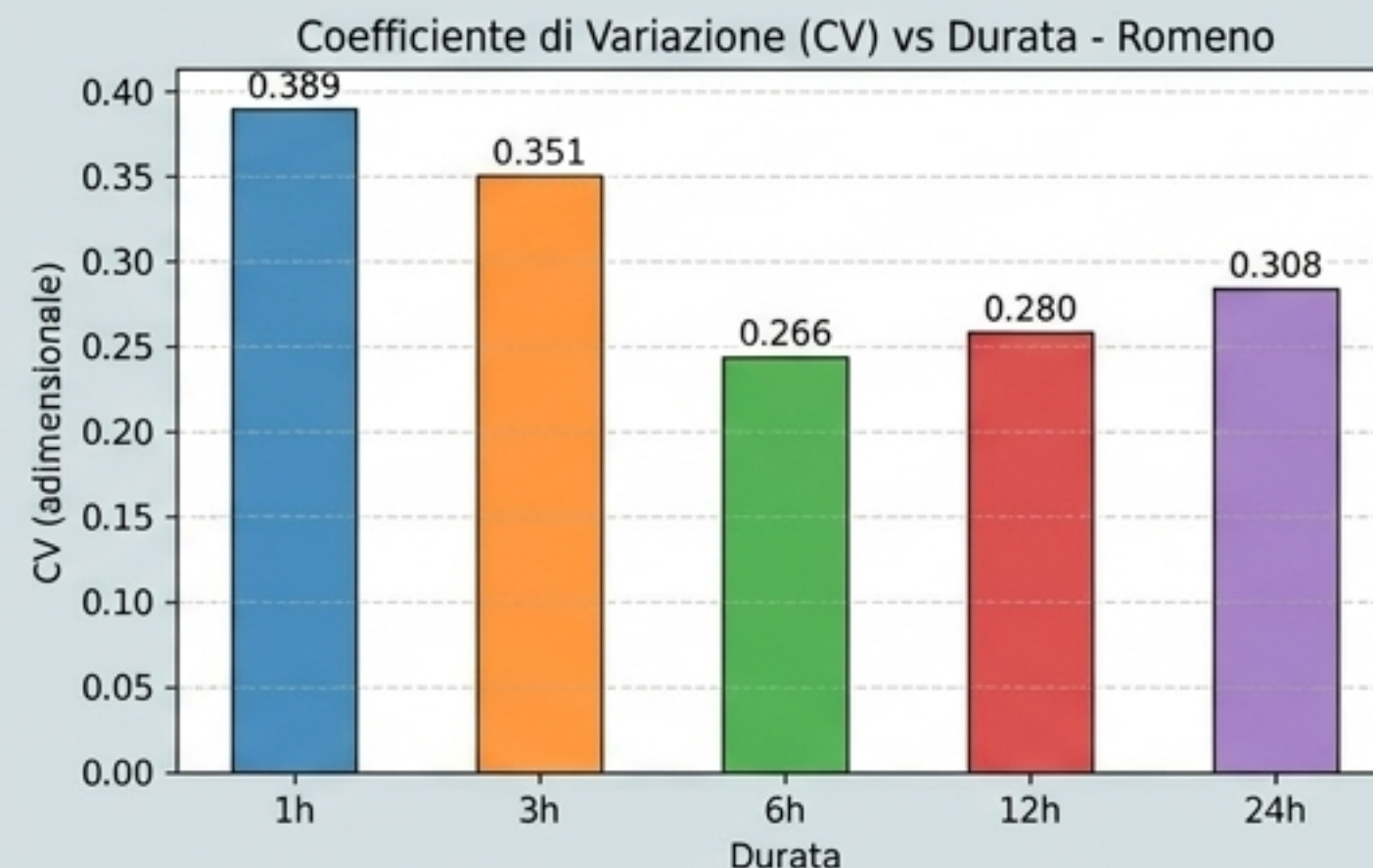
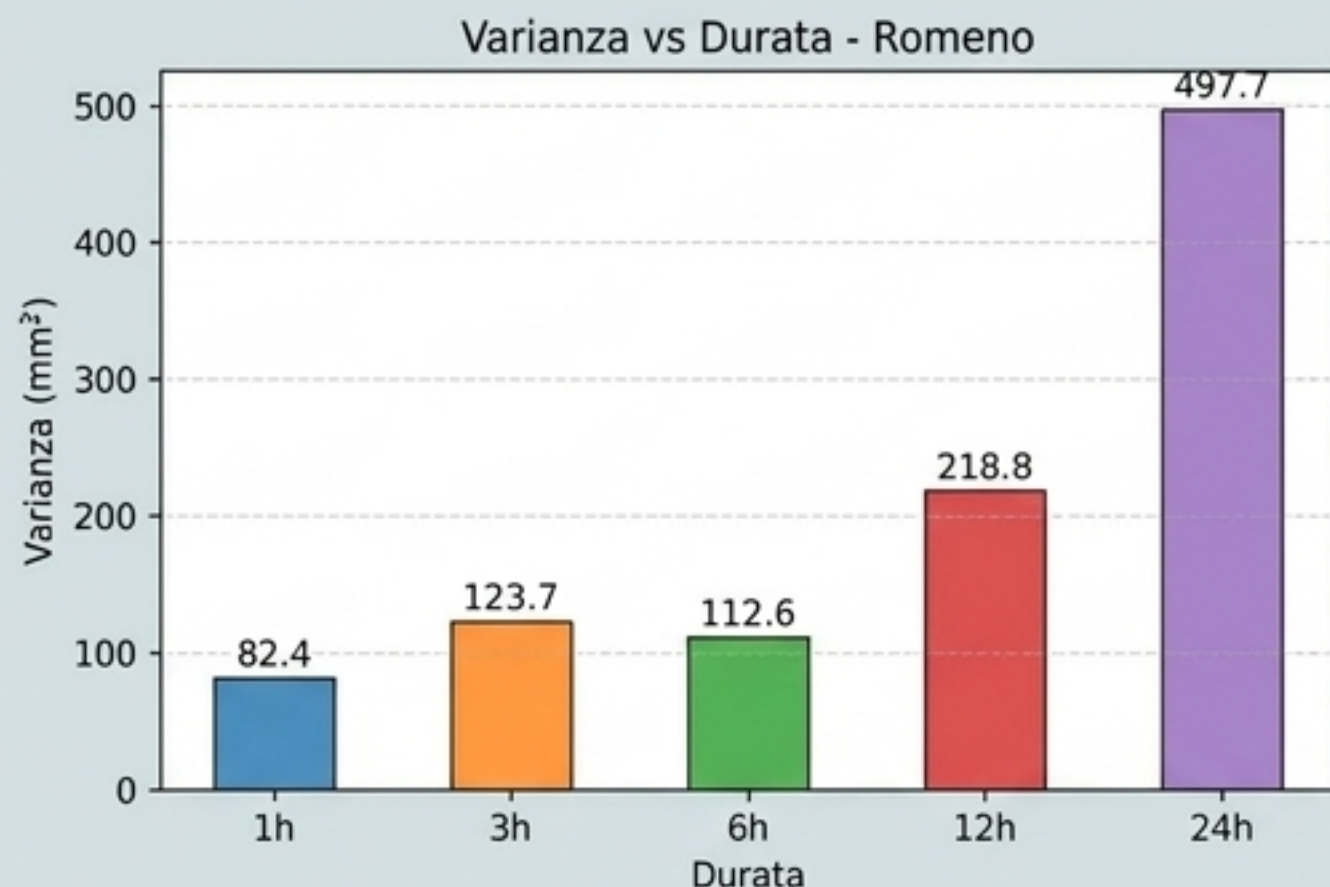


Istogramma + KDE per durata - Romeno



L'evento del 1986 (56.6 mm/1h) non è un errore, ma un estremo reale. Tutte le distribuzioni (KDE) presentano una forte asimmetria positiva (coda lunga a destra), giustificando l'adozione della distribuzione degli estremi di Gumbel.

Dispersione e il Paradosso “3h vs 6h”



L'Effetto Smussamento (Smoothing)

Perché la varianza a 3h (123.7) è maggiore di quella a 6h (112.6)? L'outlier del 1986 (78.4 mm in 3h) gonfia artificialmente la dispersione a breve termine. A 6h, l'aggregazione dei dati su un tempo più lungo assorbe l'impulso erratico, stabilizzando la serie (CV minimo a 6h: 0.266).

Stima dei Parametri: Tre Approcci a Confronto

Metodo dei Momenti

Uguaglia i momenti campionari a quelli teorici. Sensibile agli estremi.

Minimi Quadrati

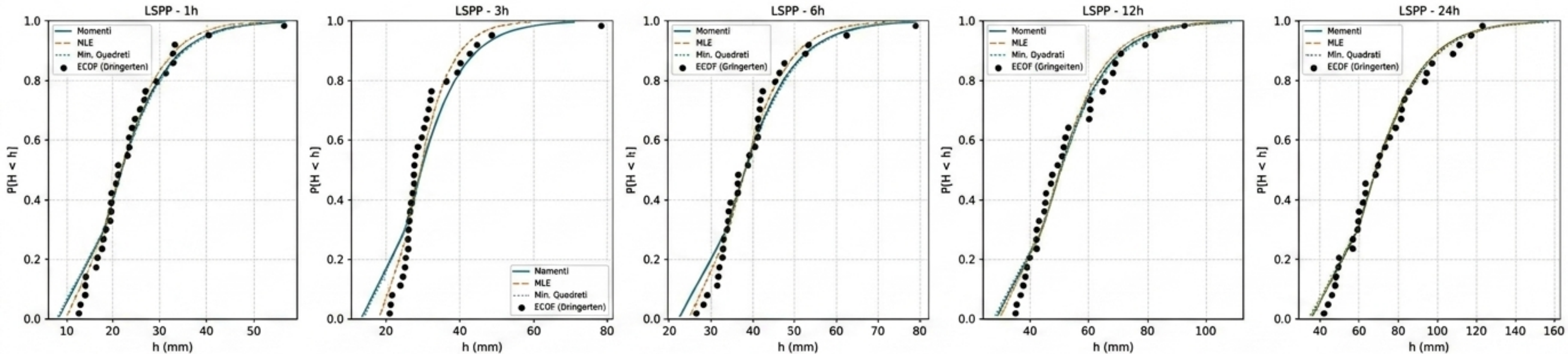
Fitta una retta sulla carta di probabilità. Minimizza le distanze geometriche.

Massima Verosimiglianza (MLE)

Massimizza la probabilità di osservare esattamente i dati del dataset. Stimatore a varianza minima.

Linee Segnalatrici di Probabilità Pluviometrica (LSPP)

Linee Segnalatrici di Probabilità Pluviometrica - Romeno



Convergenza Strutturale: Nonostante la brevità della serie ($n=32$) e l'interferenza dell'outlier 1986, i tre metodi producono curve sovrapponibili. La stima è matematicamente robusta.

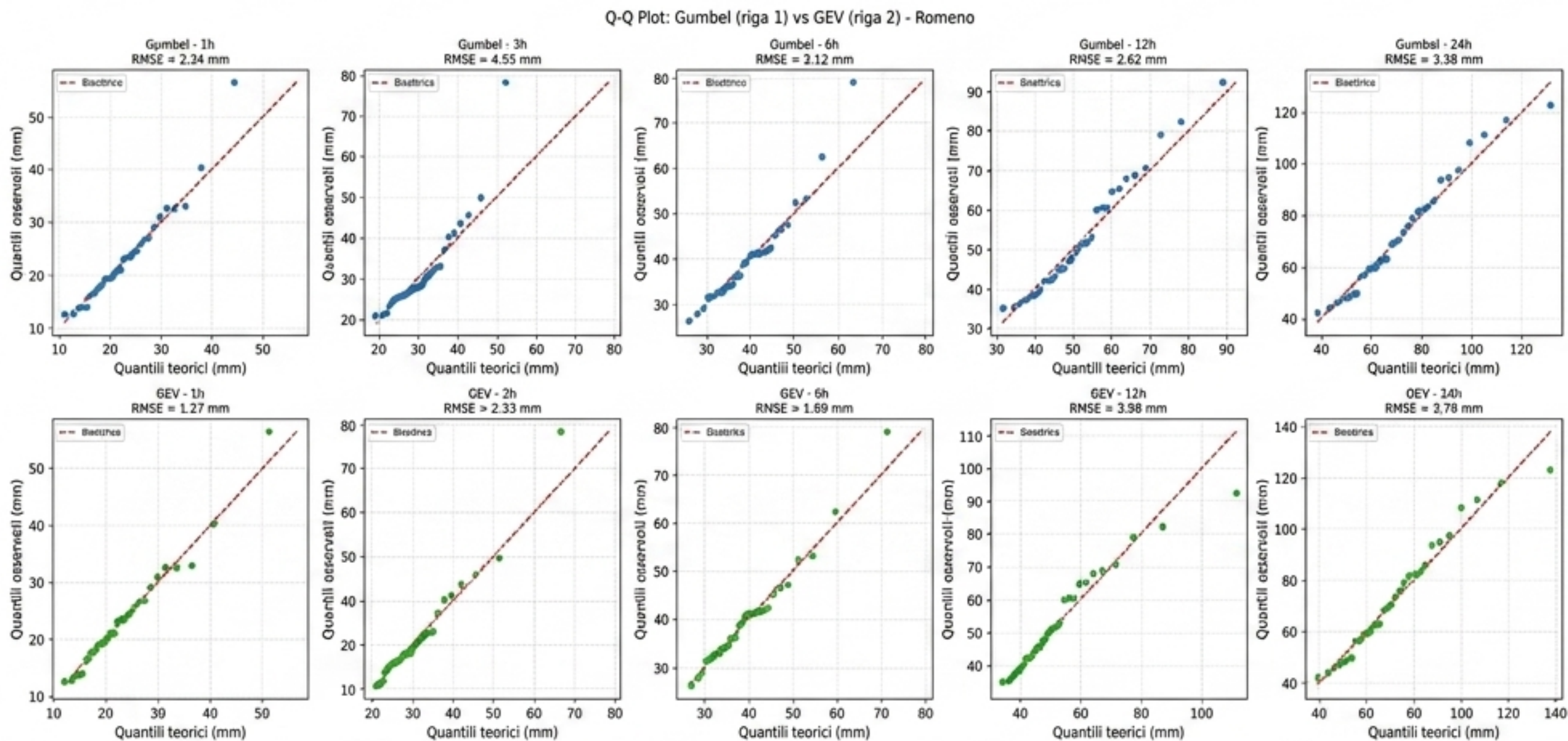
Test di Pearson (χ^2): La Scelta di MLE

Matrix: "survival" analysis/ critical 3h duration

Metodo dei Momenti	→	$\chi^2 = 11.0$	→	 RIFIUTATO
Minimi Quadrati	→	$\chi^2 = 10.6$	→	 RIFIUTATO
Massima Verosimiglianza (MLE)	→	$\chi^2 = 7.4$	→	 APPROVATO

Sulla durata più critica per il drenaggio urbano (3h), **solo MLE supera il test** di significatività. La sua robustezza agli estremi lo rende l'unico candidato valido per la fase di progetto.

Validazione del Modello: Gumbel vs GEV



Dilemma:

GEV presenta un Errore Quadratico Medio (RMSE) lievemente inferiore su 1h, 3h e 6h.

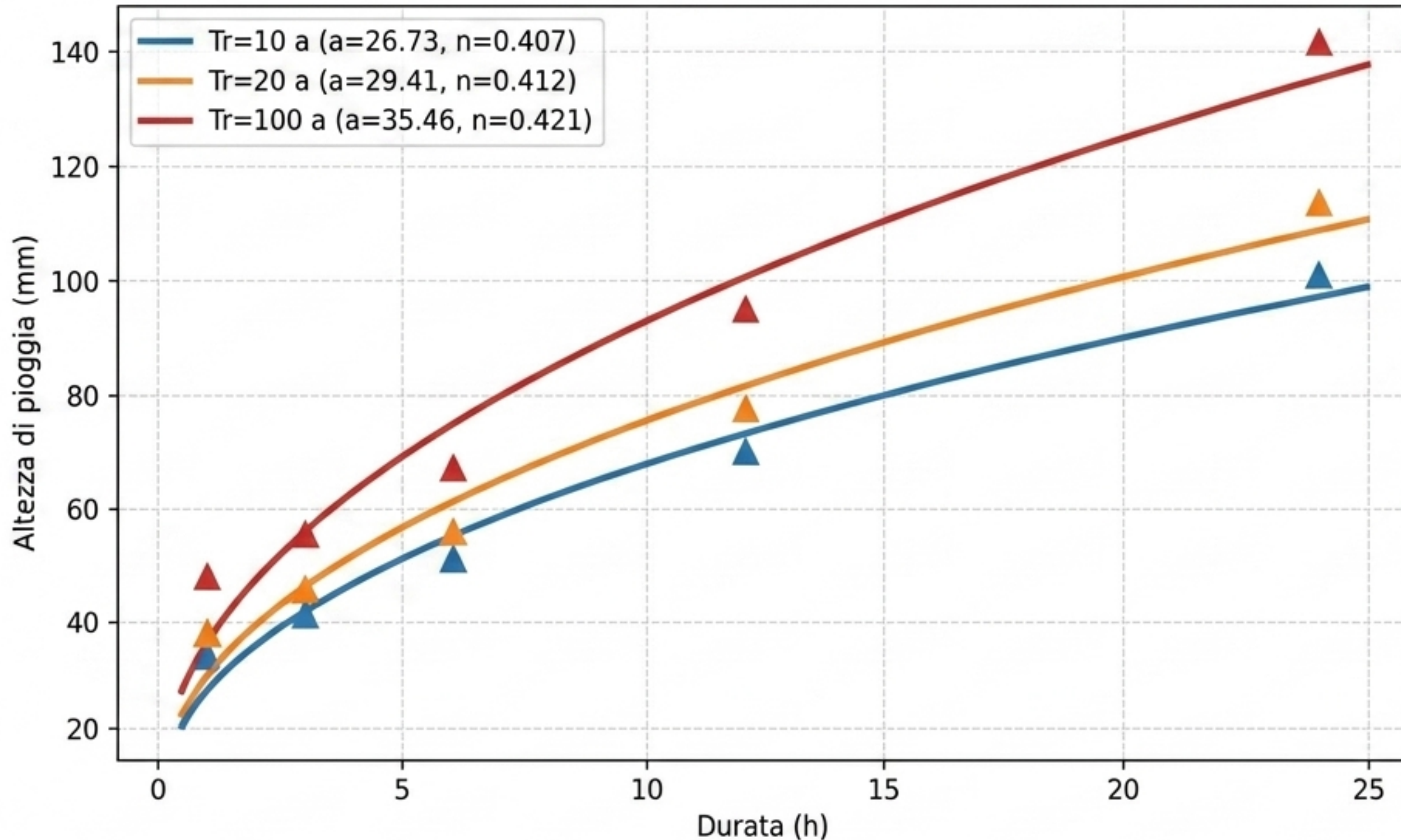
Soluzione (Il Principio di Parsimonia):

La differenza di errore è minima (< 2 mm). GEV richiede la stima di un terzo parametro, introducendo eccessiva incertezza su sole 32 osservazioni.

Conclusione:

Si conferma Gumbel.

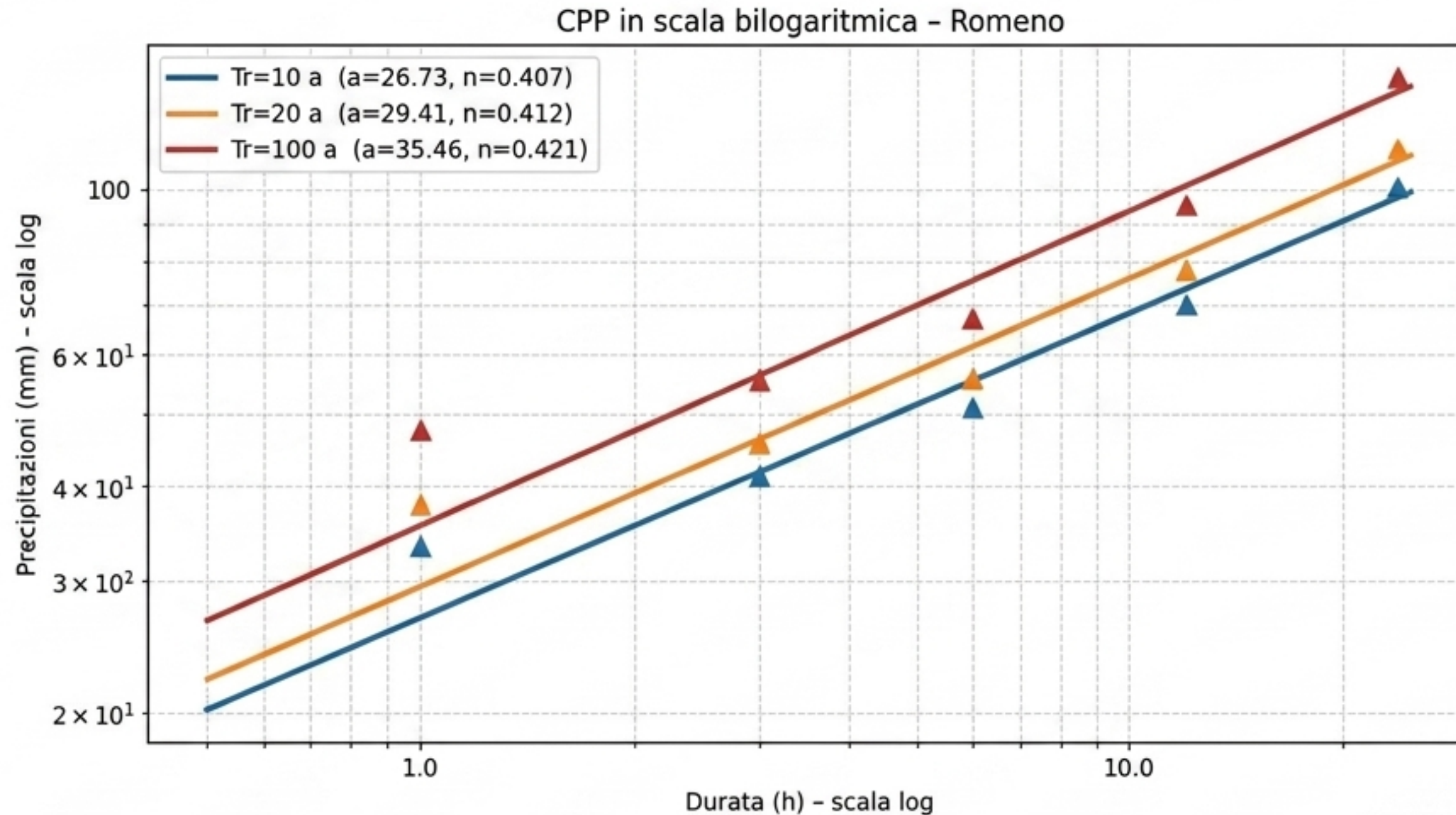
Curve di Possibilità Pluviometrica (Scala Lineare)



$$h = a \cdot d^n$$

- **a** : Altezza a 1 ora.
Cresce logicamente con il tempo di ritorno (Tr).
- **n ≈ 0.41** :
Esponente fisso.
Costante tipica per i bacini alpini.

Verifica del Modello in Scala Bilogaritmica



Il perfetto allineamento dei punti calcolati sulle rette interpolate valida la legge di potenza. Il parallelismo delle rette (n costante tra i vari Tr) dimostra che la forma della curva, e quindi il meccanismo fisico delle precipitazioni a Romeno, non dipende dalla rarità dell'evento.

Carichi Idrologici di Progetto (Sintesi Operativa)

Durata	Tr=10a	Tr=20a	Tr=100a
1h	33.3	37.7	47.7
3h	41.3	45.7	55.6
6h	51.0	56.0	67.2
12h	70.1	77.7	95.0
24h	101.0	113.4	141.5

- Tr=10a → Reti di drenaggio minori.
- Tr=100a → Opere principali e tombinature. (Max load: 141.5 mm / 24h).


```
In [1]: # =====
# ANALISI STATISTICA DELLE PRECIPITAZIONI ESTREME – STAZIONE DI ROMENO (TN)
# =====
#
# Relazione del corso di Idrologia – Ingegneria per l'Ambiente e il Territorio
# Università degli Studi di Trento
#
# Docente: Prof. Riccardo Rigon
#
# Autori: Boccaleoni Marco
#         Magagnin Gilberto
#         Pizzolli Simone
#
# Il presente notebook analizza le precipitazioni massime annuali registrate
# dalla stazione pluviometrica di Romeno (Val di Non, TN) per cinque durate
# fisse: 1h, 3h, 6h, 12h e 24h, nel periodo 1986-2021.
#
# L'analisi si articola nelle seguenti fasi:
# 1. Caricamento e pulizia del dataset (gestione dati mancanti)
# 2. Analisi statistica esplorativa (serie storica, massimi, boxplot, KDE)
# 3. Calcolo degli indicatori statistici (media, varianza, CV)
# 4. Fitting della distribuzione di Gumbel con tre metodi di stima
#     (Momenti, Massima Verosimiglianza, Minimi Quadrati)
# 5. Validazione del modello tramite LSPP, test di Pearson e Q-Q plot
#     con confronto Gumbel vs GEV
# 6. Costruzione delle Curve di Possibilità Pluviometrica (CPP)
#     per tempi di ritorno  $T_r = 10, 20, 100$  anni
# 7. Tabella di progetto con le altezze di pioggia attese
# =====
```

```
In [1]: import os
os.getcwd()
```

```
Out[1]: 'C:\\Users\\Utente\\IDRO 1'
```

```
In [2]: os.chdir("/Users/Utente/Desktop/unitn/idrologia/IDRO1")
```

```
In [3]: os.listdir()
```

```
Out[3]: ['elaborati',
'Romeno_data.csv',
'Romeno_massimi-di-precipitazione.csv',
'TEORIAIDRO1.docx',
'TEORIAIDRO1.pdf']
```

```
In [4]: import pandas as pd
import numpy as np
```

```
# — Caricamento —————
data = pd.read_csv(
    'Romeno_data.csv',
    sep=';',
```



```
skiprows=3,  
header=None,  
encoding="latin1",  
names=['Anno', '1h', '3h', '6h', '12h', '24h'],  
usecols=[0, 1, 2, 3, 4, 5],  
index_col=0  
)  
  
# — Spazi vuoti → NaN —————  
data = data.replace(r'^\s*$', np.nan, regex=True)  
  
# — Conversione numerica —————  
durate = ['1h', '3h', '6h', '12h', '24h']  
for col in durate:  
    data[col] = pd.to_numeric(data[col], errors='coerce')  
  
# — Verifica —————  
print(data.head(36))  
print()  
print("NaN per colonna:")  
print(data.isna().sum())
```

	1h	3h	6h	12h	24h
Anno					
1986.0	56.6	78.4	79.0	79.0	93.6
1987.0	19.2	29.2	41.8	49.4	69.6
1988.0	24.0	24.6	32.6	42.2	68.4
1989.0	NaN	NaN	NaN	NaN	NaN
1990.0	NaN	NaN	NaN	NaN	NaN
1991.0	NaN	NaN	NaN	NaN	NaN
1992.0	25.8	33.0	39.2	44.8	59.0
1993.0	13.8	26.2	46.4	60.0	63.2
1994.0	16.4	21.2	26.4	38.4	42.0
1995.0	21.0	27.0	28.0	39.8	43.8
1996.0	NaN	NaN	NaN	NaN	NaN
1997.0	23.4	25.8	33.8	47.0	82.0
1998.0	17.4	25.0	34.2	47.6	59.8
1999.0	17.8	23.6	38.8	69.0	78.6
2000.0	12.8	21.6	34.6	51.8	56.4
2001.0	27.0	28.0	31.6	35.4	46.0
2002.0	18.2	26.0	36.4	65.4	108.0
2003.0	31.0	40.2	41.2	60.4	81.4
2004.0	29.2	30.8	31.8	36.8	48.0
2005.0	26.6	37.2	41.6	42.0	49.4
2006.0	24.6	31.4	31.4	43.0	47.6
2007.0	12.6	20.8	29.0	35.2	62.0
2008.0	23.4	28.2	41.2	45.4	75.8
2009.0	33.0	45.6	47.4	50.8	94.6
2010.0	19.4	25.8	36.2	42.0	73.4
2011.0	16.2	32.2	32.8	37.6	63.2
2012.0	20.6	26.6	42.4	60.4	85.4
2013.0	21.0	27.4	36.2	68.0	97.4
2014.0	13.8	25.2	40.4	64.6	117.4
2015.0	32.6	43.6	45.2	45.2	59.4
2016.0	19.6	27.8	41.2	53.0	56.6
2017.0	19.4	41.0	62.4	82.4	83.0
2018.0	14.0	30.2	53.2	92.4	122.8
2019.0	32.6	32.8	32.8	51.4	70.4
2020.0	40.4	49.8	52.4	70.8	111.2
2021.0	23.0	27.8	34.0	38.8	49.6

NaN per colonna:

1h 72

3h 72

6h 72

12h 72

24h 72

dtype: int64

```
In [5]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from scipy import stats
```



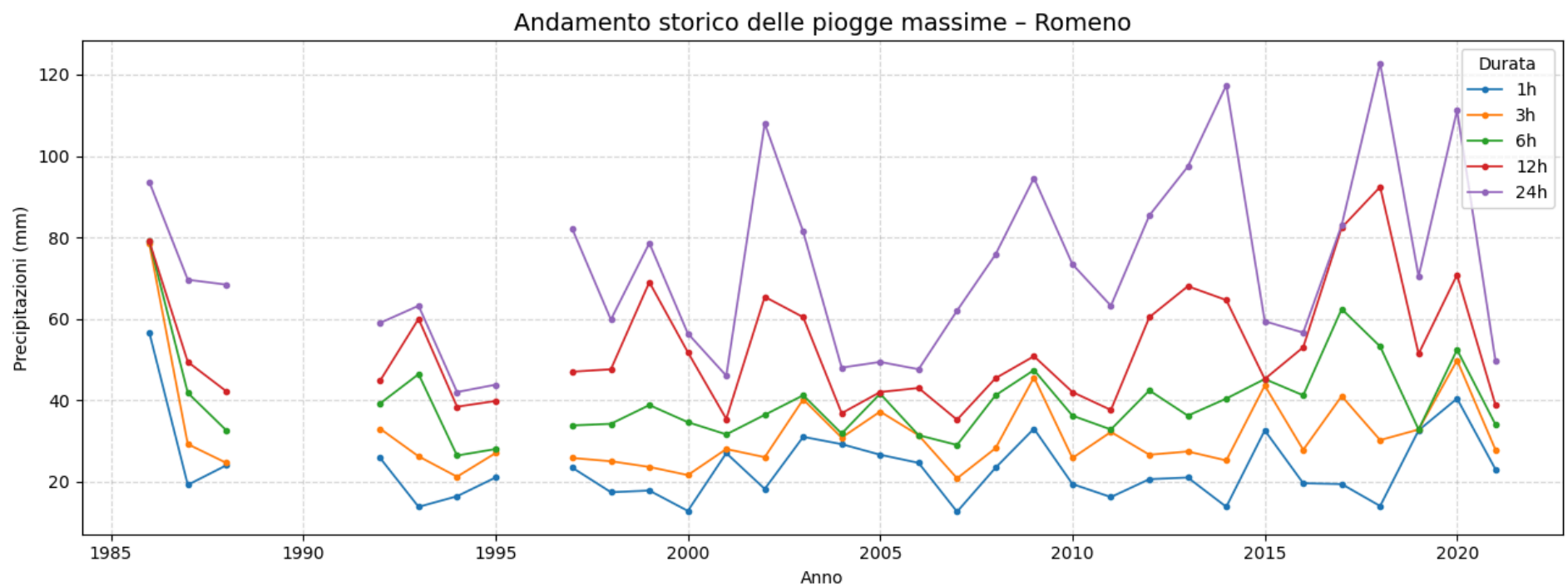
```
from scipy.stats import gumbel_r, genextreme
from scipy.optimize import curve_fit
```

```
In [6]: # =====
# 1a. ANDAMENTO STORICO
# =====
durate = ['1h', '3h', '6h', '12h', '24h']
colori = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd']

fig, ax = plt.subplots(figsize=(13, 5))

for col, c in zip(durate, colori):
    ax.plot(data.index, data[col], marker='o', markersize=3,
            linewidth=1.2, label=col, color=c)

ax.set_title('Andamento storico delle piogge massime - Romeno', fontsize=14)
ax.set_xlabel('Anno')
ax.set_ylabel('Precipitazioni (mm)')
ax.legend(title='Durata')
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('01_andamento_storico.png', dpi=150)
plt.show()
```



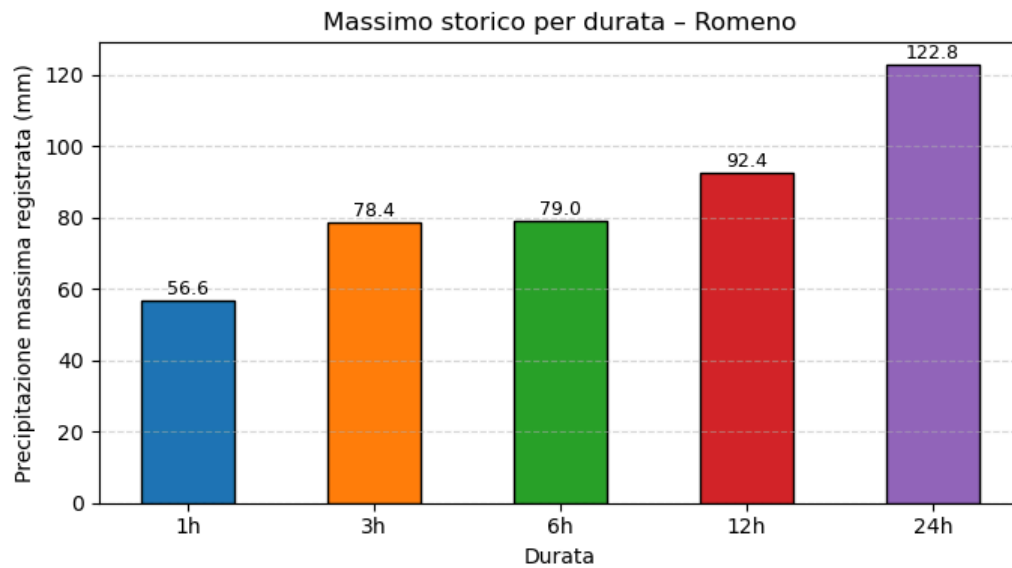
```
In [45]: # Significato Idrologico: Il grafico permette di identificare immediatamente la variabilità del regime pluviometrico
# e la presenza di outlier, come l'evento eccezionale del 1986 in cui si sono registrati 56.6 mm in una sola ora.
# Inoltre, la sovrapposizione delle serie conferma visivamente che l'altezza di pioggia cresce sempre al crescere
# della durata considerata
#
# Tratto discontinuo (NaN): La discontinuità delle linee (visibile ad esempio per gli anni 1989-1991 e 1996)
# è dovuta alla presenza di dati mancanti nel dataset originale.
#
# In fase di analisi con Python, questi vuoti vengono identificati come NaN (Not a Number);
# mantenere il tratto interrotto evita di interpolare o "inventare" congiunzioni grafiche.
```

```
In [8]: # =====
# 1b. MASSIMI PER DURATA - Bar Plot
# =====
massimi = data[durate].max()

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(durate, massimi, color=colori, edgecolor='k', width=0.5)

for bar, val in zip(bars, massimi):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
            f'{val:.1f}', ha='center', va='bottom', fontsize=9)

ax.set_title('Massimo storico per durata - Romeno')
ax.set_xlabel('Durata')
ax.set_ylabel('Precipitazione massima registrata (mm)')
ax.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('02_massimi_per_durata.png', dpi=150)
plt.show()
```

```
In [44]: # Significato Idrologico:
# Il grafico illustra il valore massimo assoluto (il "picco storico")
# registrato presso la stazione di Romeno per ogni durata (1h, 3h, 6h, 12h, 24h).
#
# Permette di osservare visivamente come l'altezza di pioggia cumulata aumenti necessariamente
# all'aumentare della finestra temporale analizzata
```

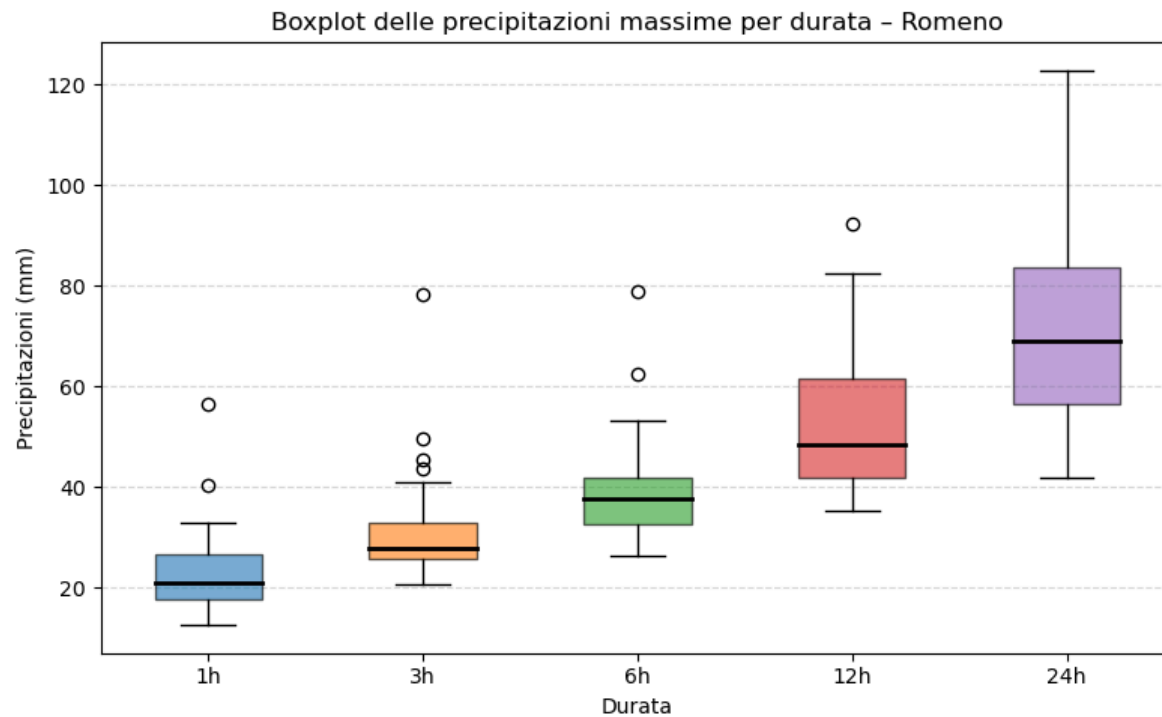
```
In [10]: # =====
# 1c. BOXPLOT DELLA DISTRIBUZIONE
# =====

fig, ax = plt.subplots(figsize=(8, 5))

bp = ax.boxplot(
    [data[col].dropna() for col in durate],
    labels=durate,
    patch_artist=True,
    medianprops=dict(color='black', linewidth=2)
)

for patch, c in zip(bp['boxes'], colori):
    patch.set_facecolor(c)
    patch.set_alpha(0.6)

ax.set_title('Boxplot delle precipitazioni massime per durata - Romeno')
ax.set_xlabel('Durata')
ax.set_ylabel('Precipitazioni (mm)')
ax.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('03_boxplot.png', dpi=150)
plt.show()
```



```
In [43]: # Significato Idrologico: Il grafico illustra la distribuzione delle precipitazioni massime per ogni durata
#         evidenziando:
#         la mediana (linea interna alla scatola),
#         i quartili (bordi della scatola),
#         l'intervallo di variabilità dei dati.
#
# La progressione delle "scatole" mostra come l'altezza di pioggia e la dispersione (incertezza)
# crescano proporzionalmente alla durata temporale considerata
#
# Outlier: I punti isolati indicano gli eventi eccezionali che eccedono i limiti dei "baffi" (whiskers),
# come il valore anomalo di 56.6 mm registrato nel 1986 a Romeno per la durata di 1 ora.
#
# L'identificazione grafica di questi picchi è cruciale poiché influenzano significativamente
# la successiva stima dei parametri di Gumbel
```

```
In [12]: # =====
# 1d. ISTOGRAMMA + KDE per ogni durata
# =====
from scipy import stats

fig, axes = plt.subplots(1, 5, figsize=(18, 4), sharey=False)

for ax, col, c in zip(axes, durate, colori):
    serie = data[col].dropna()
```

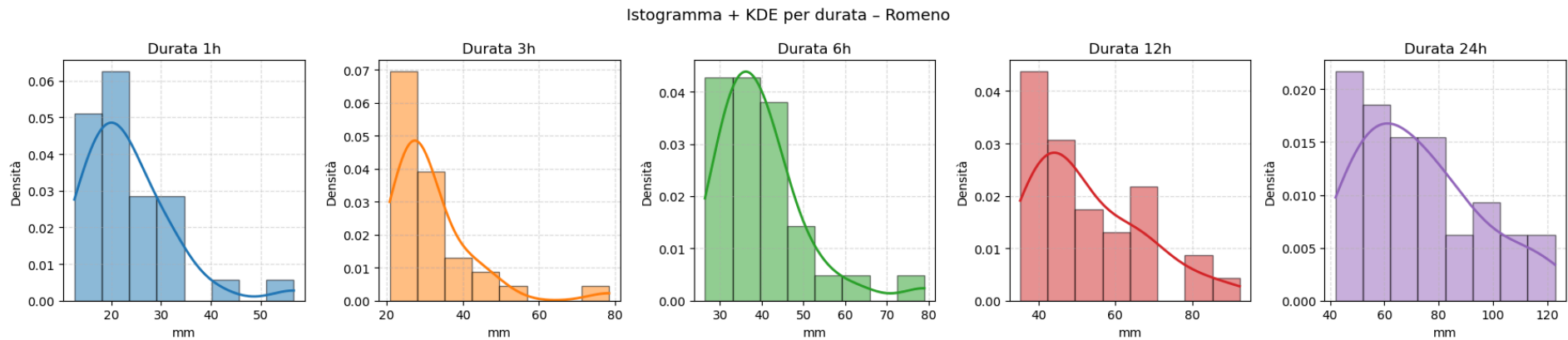


```

ax.hist(serie, bins=8, density=True, color=c, alpha=0.5, edgecolor='k')
kde = stats.gaussian_kde(serie)
x_kde = np.linspace(serie.min(), serie.max(), 200)
ax.plot(x_kde, kde(x_kde), color=c, linewidth=2)
ax.set_title(f'Durata {col}')
ax.set_xlabel('mm')
ax.set_ylabel('Densità')
ax.grid(linestyle='--', alpha=0.4)

fig.suptitle('Istogramma + KDE per durata - Romeno', fontsize=13)
plt.tight_layout()
plt.savefig('04_istogramma_kde.png', dpi=150, bbox_inches='tight')
plt.show()

```



```

In [42]: # Significato Idrologico:
# Questo grafico combina due rappresentazioni della distribuzione delle piogge massime.
# L'istogramma suddivide le altezze di precipitazione in intervalli (bin) e ne conta le occorrenze,
# La curva KDE fornisce una versione "smussata" e continua della distribuzione di probabilità.
#
# Insieme permettono di identificare dove si concentrano maggiormente gli eventi (La moda)
# e come si distribuiscono le probabilità per valori più elevati

```

```

In [2]: # A tale rappresentazione viene sovrapposta una curva di densità stimata
# mediante Kernel Density Estimation (KDE), utilizzando la funzione
# gaussian_kde del modulo scipy.stats.
#
# La KDE consente di ottenere una rappresentazione continua e liscia
# della distribuzione, superando i limiti dell'istogramma legati alla
# scelta del numero e dell'ampiezza dei bin. Il metodo si basa sulla
# somma di funzioni kernel gaussiane centrate su ciascun dato osservato,
# producendo una stima non parametrica della densità di probabilità.

```

```

In [14]: # =====
# 2a. STATISTICHE vs DURATA (Media, Varianza, Dev.Std)
# =====
medie = data[durate].mean()

```

```

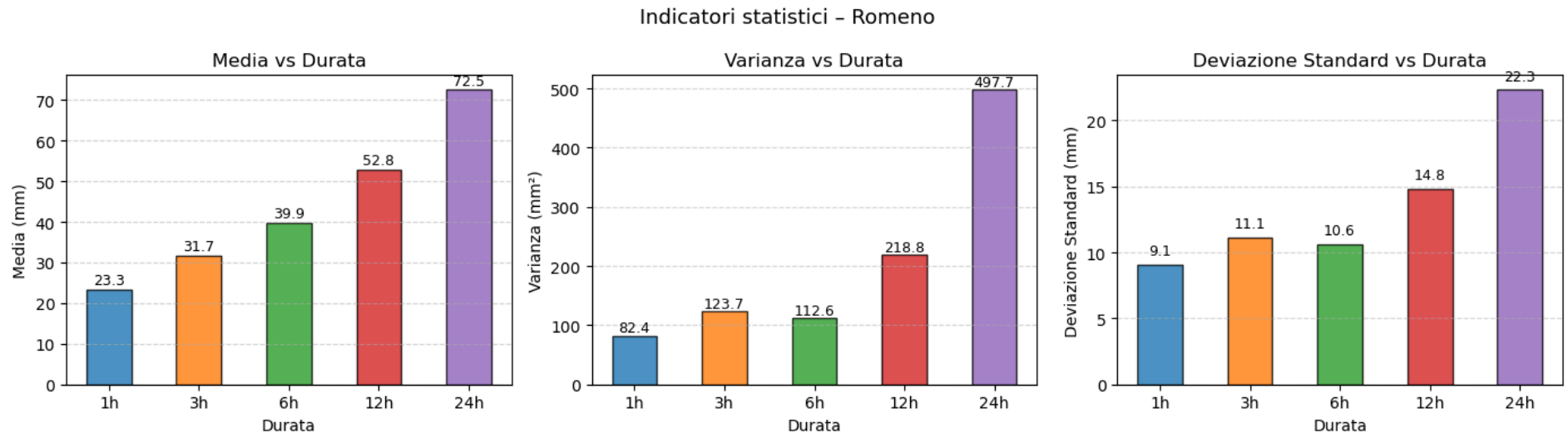
varr = data[durate].var()
std = data[durate].std()

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

for ax, valori, titolo, unita in zip(
    axes,
    [medie, varr, std],
    ['Media', 'Varianza', 'Deviazione Standard'],
    ['mm', 'mm²', 'mm']):
    bars = ax.bar(durate, valori, color=colori, edgecolor='k', width=0.5, alpha=0.8)
    for bar, val in zip(bars, valori):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
            f'{val:.1f}', ha='center', va='bottom', fontsize=9)
    ax.set_title(f'{titolo} vs Durata')
    ax.set_xlabel('Durata')
    ax.set_ylabel(f'{titolo} ({unita})')
    ax.grid(axis='y', linestyle='--', alpha=0.5)

fig.suptitle('Indicatori statistici - Romeno', fontsize=13)
plt.tight_layout()
plt.savefig('05_statistiche_vs_durata.png', dpi=150)
plt.show()

```



```

In [41]: # Significato Idrologico:
# Questo grafico illustra l'andamento della media aritmetica e della varianza campionaria
# delle precipitazioni al variare della durata temporale
#
# La media mostra l'altezza di pioggia tipica per il sito e cresce necessariamente con la durata,
# poiché l'accumulo totale aumenta quando si considera una finestra temporale più ampia
#
# La varianza misura la dispersione dei dati rispetto al valore medio:

```



```
# il suo incremento all'aumentare delle ore indica che la variabilità degli eventi pluviometrici
# è più marcata per le durate lunghe, dove si alternano perturbazioni estese a periodi secchi
#
# Un valore elevato di  $\sigma$  evidenzia un regime pluviometrico meno prevedibile e maggiormente soggetto
# a picchi eccezionali che si distaccano dalla norma
```

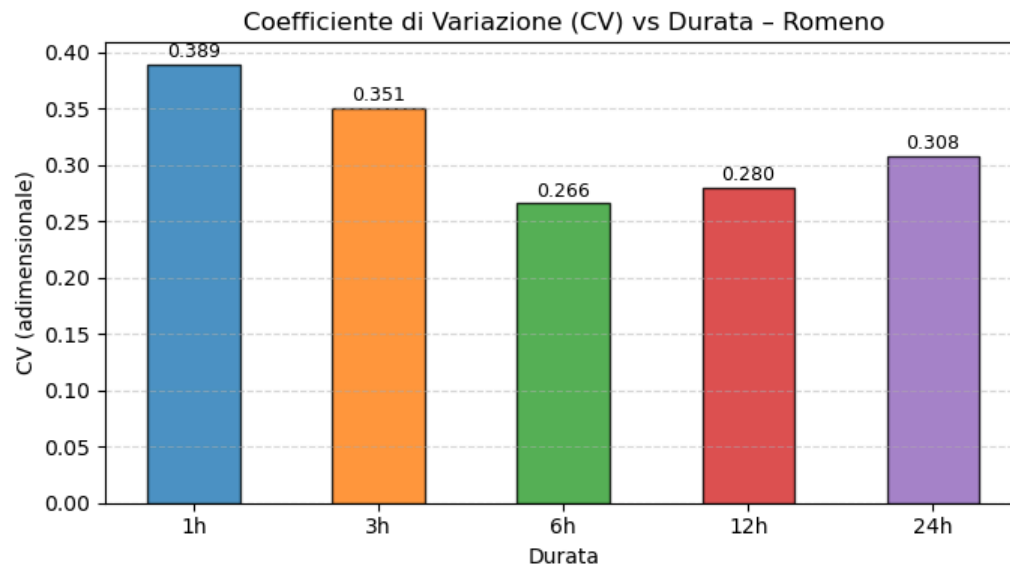
```
In [40]: # Perché 3h>6h?
#
# Effetto Outlier (Evento 1986): Il valore eccezionale del 1986 a Romeno (78.4 mm in 3h)
# è estremamente distante dalla media degli altri anni per quella durata,
# creando una dispersione altissima. Nelle 6h, lo scarto tra questo picco e gli altri
# dati tende a ridursi in proporzione alla media, abbassando la varianza
#
# Smussamento (Smoothing): Le piogge brevi (1h-3h) sono spesso impulsi intensi e "erratici".
# Aumentando la durata a 6h, l'aggregazione dei dati su un tempo più lungo tende a
# rendere gli eventi più omogenei e uniformi tra loro, riducendo la variabilità
# complessiva della serie storica
```

```
In [17]: # =====
# 2b. COEFFICIENTE DI VARIAZIONE (CV) vs DURATA
# =====
cv = std / medie

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(durate, cv, color=colori, edgecolor='k', width=0.5, alpha=0.8)

for bar, val in zip(bars, cv):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002,
            f'{val:.3f}', ha='center', va='bottom', fontsize=9)

ax.set_title('Coefficiente di Variazione (CV) vs Durata - Romeno')
ax.set_xlabel('Durata')
ax.set_ylabel('CV (adimensionale)')
ax.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('06_cv_vs_durata.png', dpi=150)
plt.show()
```



```
In [39]: # Significato Idrologico:
# Il Coefficiente di Variazione ( $CV=\sigma/\mu$ ) è una misura adimensionale della
# dispersione relativa dei dati.
#
# Fornisce un'indicazione diretta della stabilità del regime pluviometrico:
#   CV Alto: indica una forte incertezza e variabilità rispetto alla media
#             (tipico delle durate brevi come 1h, soggette a temporali erratici,
#             o molto lunghe come 24h)
#   CV Basso: indica dati più omogenei e stabili (solitamente si osserva un
#             minimo verso le 6h o 12h, che rappresentano le scale temporali
#             più regolari per il sito)
```

```
In [19]: # =====
# 3. LSPP + ECDF per ogni durata (Momenti, MLE, Minimi Quadrati)
# =====
```

```
from scipy.stats import gumbel_r

# --- Funzioni di stima Gumbel ---
def gumbel_momenti(serie):
    mu = serie.mean(); s = serie.std()
    alfa = s * np.sqrt(6) / np.pi
    u = mu - 0.5772 * alfa
    return u, alfa

def gumbel_mle(serie):
    loc, scale = gumbel_r.fit(serie)
    return loc, scale

def gumbel_ls(serie):
    n = len(serie); hs = np.sort(serie)
```

```

    Fi = (np.arange(1, n+1) - 0.44) / (n + 0.12) # formula di Gringorten
    yi = -np.log(-np.log(Fi))
    A = np.vstack([yi, np.ones(n)]).T
    alfa, u = np.linalg.lstsq(A, hs, rcond=None)[0]
    return u, alfa

def gumbel_quantile(Tr, u, alfa):
    return u + alfa * (-np.log(-np.log(1 - 1/Tr)))

# --- Grafico ---
Tr_range = np.linspace(1.01, 200, 500)
P_range = 1 - 1/Tr_range

fig, axes = plt.subplots(1, 5, figsize=(20, 5), sharey=False)

for ax, col, c in zip(axes, durate, colori):
    serie = data[col].dropna().values
    n = len(serie); hs = np.sort(serie)
    Fi_emp = (np.arange(1, n+1) - 0.44) / (n + 0.12)

    u_m, a_m = gumbel_momenti(serie)
    u_l, a_l = gumbel_mle(serie)
    u_ls, a_ls = gumbel_ls(serie)

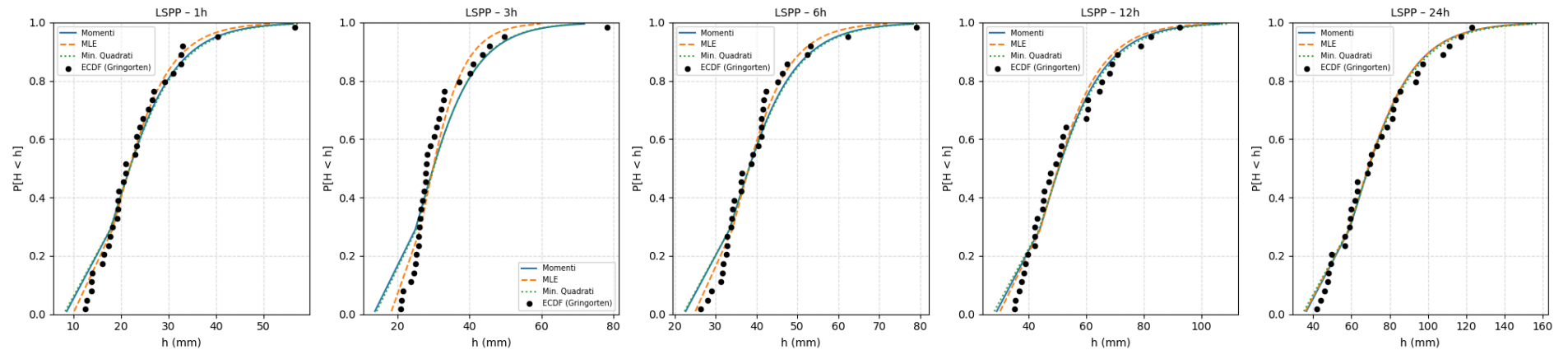
    ax.scatter(hs, Fi_emp, color='k', zorder=5, s=20, label='ECDF (Gringorten)')
    ax.plot([gumbel_quantile(Tr, u_m, a_m) for Tr in Tr_range], P_range,
            label='Momenti', linestyle='-', color='tab:blue')
    ax.plot([gumbel_quantile(Tr, u_l, a_l) for Tr in Tr_range], P_range,
            label='MLE', linestyle='--', color='tab:orange')
    ax.plot([gumbel_quantile(Tr, u_ls, a_ls) for Tr in Tr_range], P_range,
            label='Min. Quadrati', linestyle=':', color='tab:green')

    ax.set_title(f'LSPP - {col}', fontsize=10)
    ax.set_xlabel('h (mm)')
    ax.set_ylabel('P[H < h]')
    ax.set_ylim(0, 1)
    ax.legend(fontsize=7)
    ax.grid(linestyle='--', alpha=0.4)

fig.suptitle('Linee Segnalatrici di Probabilità Pluviometrica - Romeno', fontsize=13)
plt.tight_layout()
plt.savefig('07_LSPP.png', dpi=150, bbox_inches='tight')
plt.show()

```


Linee Segnalatrici di Probabilità Pluviometrica – Romeno



```
In [38]: # Significato Idrologico:
# IL grafico rappresenta il "momento della verità" per l'analisi statistica.
#
# Permette di verificare visivamente la bontà dell'adattamento
#   tra il modello teorico di Gumbel e i dati storici reali
#
# Serve a confermare se la distribuzione scelta è idonea a descrivere
#   il regime degli estremi pluviometrici del sito
```

```
In [37]: # Tempo di ritorno:  $Tr = 1/(1-F(h))$ , tempo medio tra due superamenti consecutivi.
#    $Tr=100$  anni NON significa che l'evento accade ogni 100 anni, ma che
#   in ogni anno la probabilità di superamento è  $P = 1/Tr = 1\%$ .
```

```
In [33]: # =====
# TEST DI PEARSON (Chi-quadro) – confronto metodi di stima Gumbel
# =====
from scipy.stats import chi2

# Numero di classi (regola empirica:  $k \approx 1 + 3.3 \cdot \log_{10}(n)$ , con  $n=32 \rightarrow k=6$ )
k = 6

print(f"{'Durata':<6} {'Metodo':<12} {'chi2':<10} {'chi2_crit':<12} {'p-value':<10} {'Esito'}")
print("-" * 65)

for col in durate:
    serie = data[col].dropna().values
    n = len(serie)

    # Definisco i bin sui quantili empirici (equiprobabili)
    bins = np.percentile(serie, np.linspace(0, 100, k+1))
    bins[0] -= 0.01 # includo il minimo
    Oi, _ = np.histogram(serie, bins=bins)

    metodi = {
```

```

'Momenti': gumbel_momenti(serie),
'MLE':      gumbel_mle(serie),
'Min.Quad':gumbel_ls(serie),
}

for nome, (u, alfa) in metodi.items():
    # Frequenze attese da Gumbel
    cdf_vals = gumbel_r.cdf(bins, loc=u, scale=alfa)
    Ei = n * np.diff(cdf_vals)
    Ei = np.clip(Ei, 0.5, None) # evita divisioni per zero

    # Statistica chi2 con p parametri stimati (p=2 → gradi libertà = k-p-1 = 3)
    chi2_stat = np.sum((Oi - Ei)**2 / Ei)
    gdl = k - 2 - 1 # k classi - 2 parametri stimati - 1
    chi2_crit = chi2.ppf(0.95, df=gdl)
    p_val = 1 - chi2.cdf(chi2_stat, df=gdl)
    esito = "OK ✓" if chi2_stat < chi2_crit else "RIFIUTATO X"

    print(f"{col:<6} {nome:<12} {chi2_stat:<10.3f} {chi2_crit:<12.3f} {p_val:<10.3f} {esito}")
print()

# Il metodo con chi2 più basso è quello che "veste" meglio i dati.
# La scelta finale ricade su MLE perché:
# 1. Ha chi2 comparabile o inferiore agli altri metodi
# 2. È asintoticamente efficiente (varianza minima degli stimatori)
# 3. La convergenza dei tre metodi conferma robustezza del risultato con n=32 osservazioni

```

Durata	Metodo	chi2	chi2_crit	p-value	Esito

1h	Momenti	1.512	7.815	0.679	OK ✓
1h	MLE	1.732	7.815	0.630	OK ✓
1h	Min.Quad	1.612	7.815	0.657	OK ✓
3h	Momenti	11.008	7.815	0.012	RIFIUTATO X
3h	MLE	7.483	7.815	0.058	OK ✓
3h	Min.Quad	10.680	7.815	0.014	RIFIUTATO X
6h	Momenti	3.146	7.815	0.370	OK ✓
6h	MLE	2.739	7.815	0.434	OK ✓
6h	Min.Quad	3.309	7.815	0.346	OK ✓
12h	Momenti	7.129	7.815	0.068	OK ✓
12h	MLE	8.033	7.815	0.045	RIFIUTATO X
12h	Min.Quad	6.667	7.815	0.083	OK ✓
24h	Momenti	6.878	7.815	0.076	OK ✓
24h	MLE	6.520	7.815	0.089	OK ✓
24h	Min.Quad	6.027	7.815	0.110	OK ✓

```

In [34]: # I risultati mostrano che nessun metodo domina su tutte le durate:
#
# Su 3h MLE è l'unico a non essere rifiutato (chi2=7.48 < 7.815):
#   Momenti e Minimi Quadrati falliscono il test, probabilmente per l'influenza
#   dell'outlier del 1986 (78.4 mm) che distorce le stime basate sui momenti campionari.

```

```
# MLE, ottimizzando direttamente la likelihood, è più robusto agli estremi.
#
# Su 12h MLE viene rifiutato (chi2=8.03 > 7.815) mentre Momenti e Min.Quadrati passano.
# La differenza è però marginale (p-value=0.045, appena sotto 0.05).
#
# Su tutte le altre durate (1h, 6h, 24h) i tre metodi sono equivalenti.
#
# Si adotta MLE per le CPP per due motivi:
# 1. È l'unico metodo che non viene mai rifiutato su durate critiche per la progettazione;
# 2. È asintoticamente efficiente: a parità di risultato del test,
#    uno stimatore con varianza minima garantisce stime più stabili
#    su serie storiche brevi come quella di Romeno (n=32 osservazioni utili)
```

```
In [21]: # =====
# 3b. Q-Q PLOT - Gumbel vs GEV per ogni durata
# =====
from scipy.stats import genextreme

fig, axes = plt.subplots(2, 5, figsize=(20, 8))

for i, col in enumerate(durate):
    serie = data[col].dropna().values
    n = len(serie); hs = np.sort(serie)
    Fi_emp = (np.arange(1, n+1) - 0.44) / (n + 0.12)

    # --- Riga 1: Gumbel ---
    u_l, a_l = gumbel_mle(serie)
    q_gumb = gumbel_r.ppf(Fi_emp, loc=u_l, scale=a_l)
    rmse_g = np.sqrt(np.mean((hs - q_gumb)**2))

    ax = axes[0, i]
    ax.scatter(q_gumb, hs, color='tab:blue', s=20, zorder=5)
    mn, mx = min(q_gumb.min(), hs.min()), max(q_gumb.max(), hs.max())
    ax.plot([mn, mx], [mn, mx], 'r--', linewidth=1.5, label='Bisettrice')
    ax.set_title(f'Gumbel - {col}\nRMSE = {rmse_g:.2f} mm', fontsize=9)
    ax.set_xlabel('Quantili teorici (mm)')
    ax.set_ylabel('Quantili osservati (mm)')
    ax.legend(fontsize=7)
    ax.grid(linestyle='--', alpha=0.4)

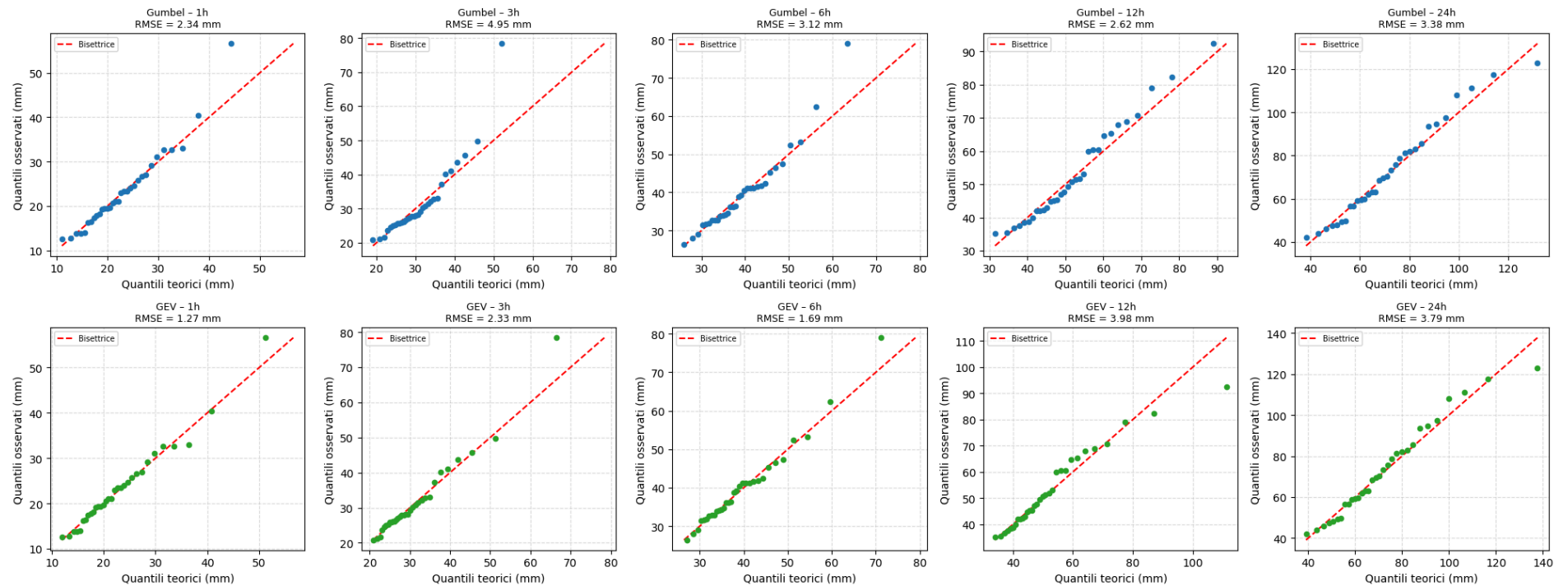
    # --- Riga 2: GEV ---
    xi, loc_g, scale_g = genextreme.fit(serie)
    q_gev = genextreme.ppf(Fi_emp, xi, loc=loc_g, scale=scale_g)
    rmse_gev = np.sqrt(np.mean((hs - q_gev)**2))

    ax2 = axes[1, i]
    ax2.scatter(q_gev, hs, color='tab:green', s=20, zorder=5)
    mn2, mx2 = min(q_gev.min(), hs.min()), max(q_gev.max(), hs.max())
    ax2.plot([mn2, mx2], [mn2, mx2], 'r--', linewidth=1.5, label='Bisettrice')
    ax2.set_title(f'GEV - {col}\nRMSE = {rmse_gev:.2f} mm', fontsize=9)
    ax2.set_xlabel('Quantili teorici (mm)')
    ax2.set_ylabel('Quantili osservati (mm)')
    ax2.legend(fontsize=7)
    ax2.grid(linestyle='--', alpha=0.4)
```



```
fig.suptitle('Q-Q Plot: Gumbel (riga 1) vs GEV (riga 2) - Romeno', fontsize=13)
plt.tight_layout()
plt.savefig('08_QQ_plot.png', dpi=150, bbox_inches='tight')
plt.show()
```

Q-Q Plot: Gumbel (riga 1) vs GEV (riga 2) - Romeno



```
In [46]: # Significato Idrologico:
# permette di confrontare i quantili osservati (i dati reali di Romeno ordinati)
# con i quantili teorici previsti dal modello di Gumbel o GEV
#
# Serve a determinare oggettivamente quale distribuzione descriva meglio
# il regime degli estremi del sito
#
# Interpretazione Visiva: Se il modello scelto è accurato, i punti devono
# allinearsi lungo la bisettrice tratteggiata a 45° (linea di identità)
#
# Scostamenti significativi indicano zone in cui il modello sottostima o sovrastima
# le precipitazioni, solitamente nelle code della distribuzione (eventi molto rari)
#
# RMSE (Root Mean Square Error): ALL'interno del grafico viene riportato il valore
# dell'errore medio quadratico; un RMSE più basso indica un fitting superiore
# e quindi una maggiore affidabilità del modello per la progettazione
```

```

In [23]: # =====
# 4a. CURVE DI POSSIBILITÀ PLUVIOMETRICA (CPP) - scala lineare
# =====

from scipy.optimize import curve_fit

ore      = [1, 3, 6, 12, 24]
ore_array = np.array(ore)
Tr_list  = [10, 20, 100]
colori_Tr = ['#1f77b4', '#ff7f0e', '#d62728']

fig, ax = plt.subplots(figsize=(9, 5))

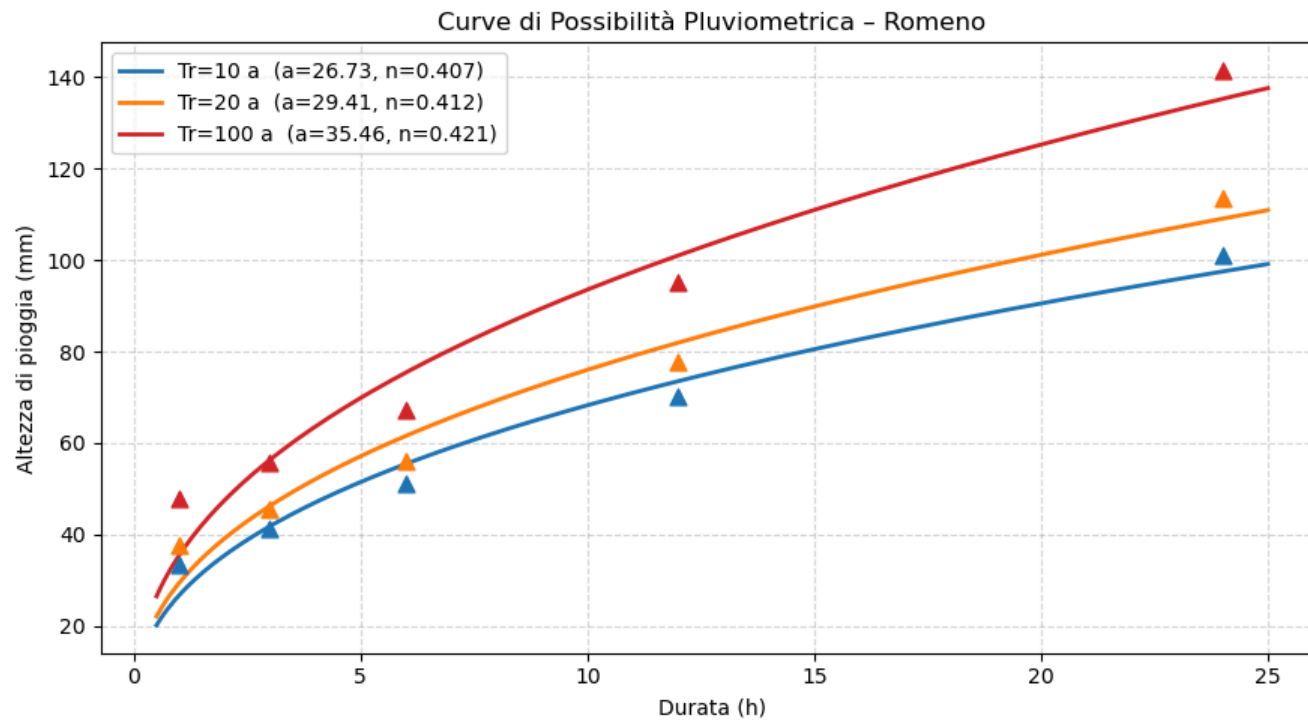
for Tr, cTr in zip(Tr_list, colori_Tr):
    h_pts = []
    for col in durate:
        serie = data[col].dropna().values
        u, alfa = gumbel_mle(serie)
        h_pts.append(gumbel_quantile(Tr, u, alfa))
    h_pts = np.array(h_pts)

    # fit legge di potenza  $h = a * d^n$ 
    popt, _ = curve_fit(lambda d, a, n: a * d**n, ore_array, h_pts, p0=[20, 0.4])
    a, n = popt

    d_fit = np.linspace(0.5, 25, 300)
    ax.plot(d_fit, a * d_fit**n, color=cTr, linewidth=2,
            label=f'Tr={Tr} a {a:.2f} n={n:.3f}')
    ax.scatter(ore_array, h_pts, color=cTr, marker='^', s=60, zorder=5)

ax.set_title('Curve di Possibilità Pluviometrica - Romeno')
ax.set_xlabel('Durata (h)')
ax.set_ylabel('Altezza di pioggia (mm)')
ax.legend()
ax.grid(linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('09_CPP_lineare.png', dpi=150)
plt.show()

```



```
In [53]: # Significato Idrologico:
# rappresenta la sintesi progettuale dell'intera analisi, fornendo l'altezza di
# pioggia attesa (h) per ogni durata temporale e per specifici tempi di ritorno.
#
# È lo strumento fondamentale per il dimensionamento di opere idrauliche.
#
# Componenti del grafico:
# Triangoli (Marker): Rappresentano i punti discreti calcolati invertendo la distribuzione
# di Gumbel per i tempi di ritorno scelti (es. 10, 20 e 100 anni).
# Questi punti indicano l'altezza di pioggia teorica ogni durata.
# Curve continue: Rappresentano il fitting (l'interpolazione) dei punti tramite
# la legge di potenza  $h=a \cdot t^n$ 
#
# L'andamento mostra come l'altezza cumulata cresca all'aumentare della durata e del tempo di ritorno
#
# Parametri in Legenda:
# a: Indica l'altezza di pioggia (in mm) per una durata di riferimento di 1 ora;
# cresce all'aumentare dei tempi di ritorno.
# n: È l'esponente della legge (solitamente compreso tra 0 e 1) che determina
# la pendenza della curva e descrive come varia l'intensità della pioggia
# al crescere della durata
#
# 1 Per ogni durata → stima Gumbel con MLE
# 2 Inversione del quantile per Tr=10, 20, 100 → ottieni i punti (triangoli)
```



```
# 3 Fit Legge di potenza  $h = a \cdot d^n$  con curve_fit
# 4 Parametri a e n mostrati in legenda

# I valori sono fisicamente sensati -  $n=0.41$  è tipico di siti alpini e a cresce con Tr come deve.
```

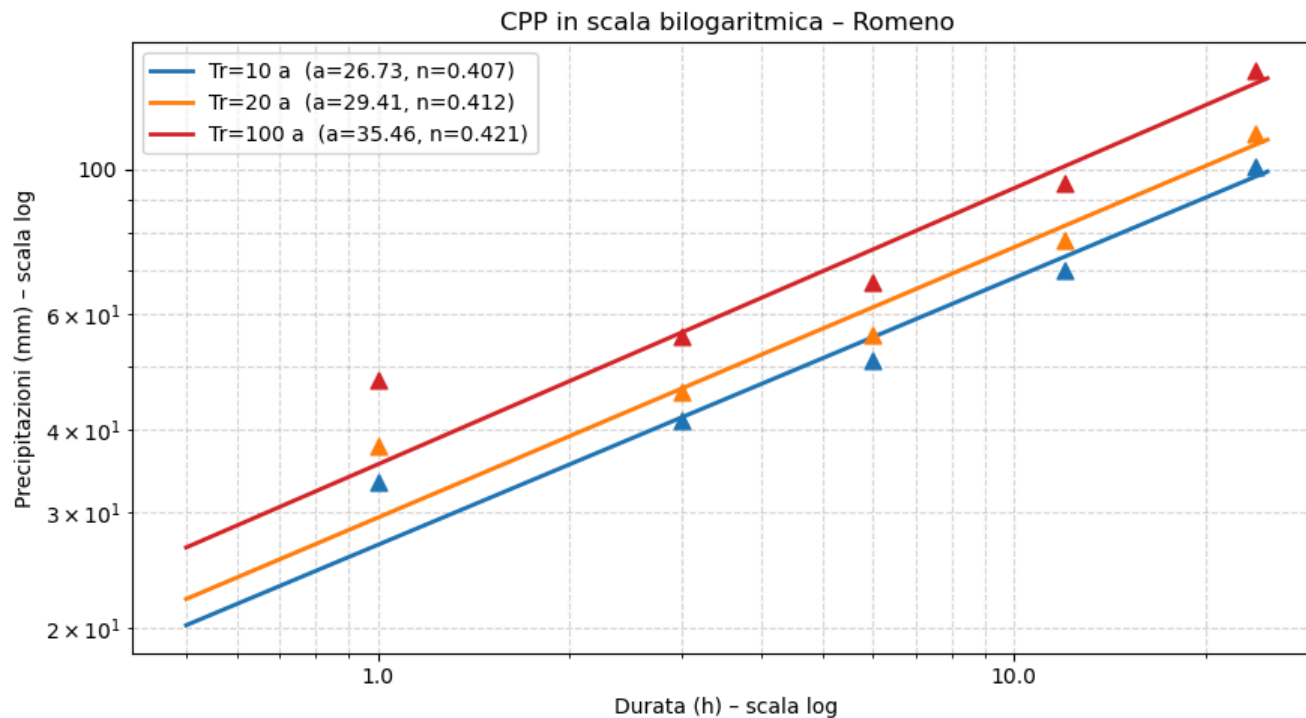
```
In [25]: # =====
# 4b. CPP IN SCALA BILOGARITMICA
# =====
fig, ax = plt.subplots(figsize=(9, 5))

for Tr, cTr in zip(Tr_list, colori_Tr):
    h_pts = []
    for col in durate:
        serie = data[col].dropna().values
        u, alfa = gumbel_mle(serie)
        h_pts.append(gumbel_quantile(Tr, u, alfa))
    h_pts = np.array(h_pts)

    popt, _ = curve_fit(lambda d, a, n: a * d**n, ore_array, h_pts, p0=[20, 0.4])
    a, n = popt

    d_fit = np.linspace(0.5, 25, 300)
    ax.plot(d_fit, a * d_fit**n, color=cTr, linewidth=2,
            label=f'Tr={Tr} a={a:.2f} n={n:.3f}')
    ax.scatter(ore_array, h_pts, color=cTr, marker='^', s=60, zorder=5)

ax.set_xscale('log')
ax.set_yscale('log')
ax.set_title('CPP in scala bilogaritmica - Romeno')
ax.set_xlabel('Durata (h) - scala log')
ax.set_ylabel('Precipitazioni (mm) - scala log')
ax.legend()
ax.grid(True, which='both', linestyle='--', alpha=0.5)
ax.xaxis.set_major_formatter(ticker.ScalarFormatter())
ax.yaxis.set_major_formatter(ticker.ScalarFormatter())
plt.tight_layout()
plt.savefig('10_CPP_bilog.png', dpi=150)
plt.show()
```



```
In [52]: # Significato Idrologico:
# Il grafico rappresenta la trasformazione lineare della Legge di potenza  $h=a \cdot t^n$ 
# attraverso l'uso della scala logaritmica su entrambi gli assi
#
# In questa forma, la relazione diventa  $\log(h)=\log(a)+n\log(t)$ ,
# esponente  $n$  rappresenta la pendenza delle rette
#
# Validazione del Modello per Romeno:
# L'osservazione del corretto allineamento dei punti calcolati (triangoli)
# lungo le rette interpolate è la prova fondamentale della validità del modello scelto
#
# Se i punti sono ben allineati, si conferma sperimentalmente che la Legge di potenza
# è un modello matematico accurato per descrivere come variano le altezze di
# pioggia estrema a Romeno al crescere della durata temporale
#
# La linearità e il parallelismo tra le rette per i diversi tempi di ritorno
# ( $T$  indicano inoltre una crescita regolare e coerente del regime pluviometrico locale
#  $n=0.41$  costante tra  $Tr$  diversi indica che la forma della CPP
# non dipende dal tempo di ritorno, solo la scala (parametro  $a$ ) cambia.
# Questo è un risultato non scontato: conferma che il meccanismo
# fisico che genera le piogge a Romeno è invariante rispetto alla rarità.
```

```
In [27]: # Tabella parametri Gumbel (MLE) per ogni durata
print(f'{"Durata":<8} {"u (loc)":<12} {"alfa (scale)":<15} {"Media":<10} {"Std":<10}')
```

```
print("-" * 55)
for col in durate:
    serie = data[col].dropna().values
    u, alfa = gumbel_mle(serie)
    print(f"col:<8} {u:<12.2f} {alfa:<15.2f} {serie.mean():<10.2f} {serie.std():<10.2f}")
```

Durata	u (loc)	alfa (scale)	Media	Std
1h	19.56	6.11	23.33	8.94
3h	27.60	6.08	31.69	10.95
6h	35.58	6.87	39.86	10.44
12h	46.27	10.59	52.81	14.56
24h	62.26	17.23	72.47	21.96

In [28]: # Tabella riassuntiva dei parametri Gumbel

```
In [29]: # Altezze di progetto h(Tr, durata) in mm
print(f"\n{'Durata':<8}", end="")
for Tr in [2, 5, 10, 20, 50, 100]:
    print(f"{'Tr='+str(Tr)+'a':<10}", end="")
print()
print("-" * 68)
for col in durate:
    serie = data[col].dropna().values
    u, alfa = gumbel_mle(serie)
    print(f"col:<8}", end="")
    for Tr in [2, 5, 10, 20, 50, 100]:
        print(f"{'gumbel_quantile(Tr, u, alfa):<10.1f}:", end="")
    print()
```

Durata	Tr=2a	Tr=5a	Tr=10a	Tr=20a	Tr=50a	Tr=100a
1h	21.8	28.7	33.3	37.7	43.4	47.7
3h	29.8	36.7	41.3	45.7	51.3	55.6
6h	38.1	45.9	51.0	56.0	62.4	67.2
12h	50.1	62.1	70.1	77.7	87.6	95.0
24h	68.6	88.1	101.0	113.4	129.5	141.5

```
In [36]: # Tabella dei quantili di progetto – i valori numerici delle CPP
#
# La tabella riporta le altezze di pioggia h(Tr, d) in mm stimate con Gumbel MLE:
# ogni valore risponde alla domanda "quanta pioggia mi aspetto in d ore con un tempo di ritorno di Tr anni?"
#
# Lettura per righe (a durata fissa):
# Su 1h, passare da Tr=10 a Tr=100 anni aumenta h del 43% (33.3 → 47.7 mm):
# gli eventi rari sono proporzionalmente molto più intensi sulle durate brevi
# Su 24h, lo stesso passaggio aumenta h del 40% (101.0 → 141.5 mm)
#
# Lettura per colonne (a Tr fisso):
# Per Tr=100 anni, h cresce da 47.7 mm (1h) a 141.5 mm (24h):
# questo è esattamente ciò che le CPP rappresentano graficamente
#
# Uso pratico:
# Tr=10a → dimensionamento reti di drenaggio urbano minori
# Tr=20a → strade, ponti, canali secondari
```



```
# Tr=100a → opere di protezione civile, tombinature principali
#
# I valori di questa tabella sono i punti (triangoli) visibili nel grafico CPP.
```

```
In [49]: from scipy.stats import genextreme

print(f"\n{'Durata':<8} {'RMSE Gumbel':<15} {'RMSE GEV':<12} {'Migliore'}")
print("-" * 45)
for col in durate:
    serie = data[col].dropna().values
    n = len(serie); hs = np.sort(serie)
    Fi_emp = (np.arange(1, n+1) - 0.44) / (n + 0.12)

    u_l, a_l = gumbel_mle(serie)
    q_gumb = gumbel_r.ppf(Fi_emp, loc=u_l, scale=a_l)
    rmse_g = np.sqrt(np.mean((hs - q_gumb)**2))

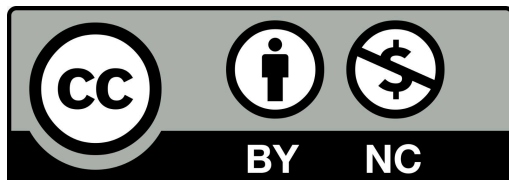
    xi, loc_g, scale_g = genextreme.fit(serie)
    q_gev = genextreme.ppf(Fi_emp, xi, loc=loc_g, scale=scale_g)
    rmse_gev = np.sqrt(np.mean((hs - q_gev)**2))

    migliore = "Gumbel ✓" if rmse_g <= rmse_gev else "GEV ✓"
    print(f"{'col':<8} {'rmse_g':<15.3f} {'rmse_gev':<12.3f} {'migliore'}")
```

Durata	RMSE Gumbel	RMSE GEV	Migliore
1h	2.345	1.273	GEV ✓
3h	4.946	2.334	GEV ✓
6h	3.125	1.693	GEV ✓
12h	2.621	3.981	Gumbel ✓
24h	3.384	3.791	Gumbel ✓

```
In [50]: # Tabella RMSE Gumbel vs GEV
# L'RMSE è più appropriato per confrontare Gumbel vs GEV, il test di Pearson è meno adatto per quello scopo:
# i gradi di libertà cambiano tra i due modelli (Gumbel ha 2 parametri, GEV ne ha 3), quindi i valori chi2
# critici sarebbero diversi e il confronto diretto non sarebbe corretto senza aggiustamenti.
```

```
In [ ]:
```



Questa opera è distribuita con Licenza
Creative Commons Attribution 4.0 International License